Payment Risk Simulator

1. System Overview

The Payment Risk Simulator is a real-time fraud detection system designed to demonstrate production-grade concurrent systems architecture. It simulates a complete payment pipeline, from transaction generation through risk evaluation to persistent audit logging, using autonomous software agents that communicate exclusively through asynchronous messages.

Most fraud detection projects stop at model training: a notebook, an accuracy number, a confusion matrix. The harder problem is running that model inside a live system where transactions arrive from multiple sources simultaneously, network calls fail, and the same payment can arrive more than once. This project builds that surrounding system.

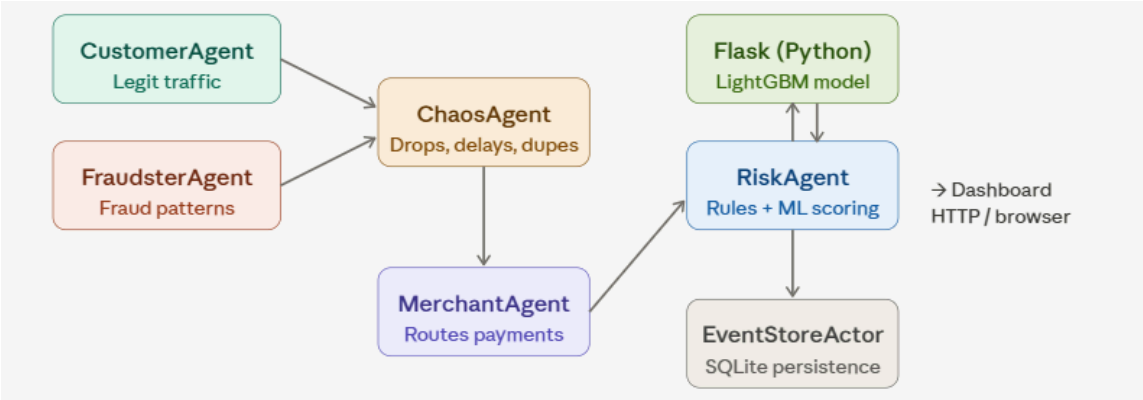
What is Akka?

Akka is a toolkit for building concurrent and distributed systems on the JVM (Java Virtual Machine). It is built around the Actor Model - a programming paradigm where the basic unit of computation is an independent entity called an actor.

- Each actor owns its private data. Nothing is shared between actors, there are no locks, no race conditions, no synchronisation primitives to manage.
- Message passing only. Actors communicate exclusively by sending typed messages to each other. An actor processes one message at a time, in order.
- Lightweight concurrency. Akka can run millions of actors on a handful of threads. The framework handles scheduling.
- Fault isolation. If one actor crashes, it does not take down the rest of the system. Supervisors can restart failed actors automatically.

The payment pipeline maps naturally onto the actor model: customers, merchants, a risk engine, a chaos agent, and a database all operate simultaneously without blocking each other. Each is an actor.

Actor Pipeline Diagram



What the System Does

- Generates realistic mixed traffic via CustomerAgent (legitimate purchases) and FraudsterAgent (velocity bursts, stolen cards, anomalous amounts) running simultaneously.
- Injects controlled network chaos such as random message delays, drops, and duplicates to test system resilience under realistic conditions.
- Scores every transaction in real time using two independent systems: a rule-based engine inside RiskAgent and a LightGBM classifier served via Flask over HTTP.

- Persists every decision to SQLite via EventStoreActor. The audit trail survives process restarts and accumulates across sessions.
- Serves a live browser dashboard via Akka HTTP, auto-refreshing every 2 seconds with colour-coded decisions and ML confidence scores.

Technology Stack

Component	Technology	Role
Actor framework	Akka Typed 2.8.5 / Scala 2.13	Concurrent agent pipeline
HTTP server	Akka HTTP 10.5.3	Serves the dashboard and JSON endpoints (/events, /stats)
ML model	LightGBM 4.6.0 (Python)	Gradient-boosted tree classifier — fraud probability scoring
ML microservice	Flask 3.1.3	Exposes the trained model as a REST endpoint over HTTP
Persistence	SQLite + JDBC	Event store (no server required, survives restarts)
Build tool	sbt 1.x	Dependency management, compilation, and local execution
Training data	Synthetic (generated)	8,000 labelled transactions

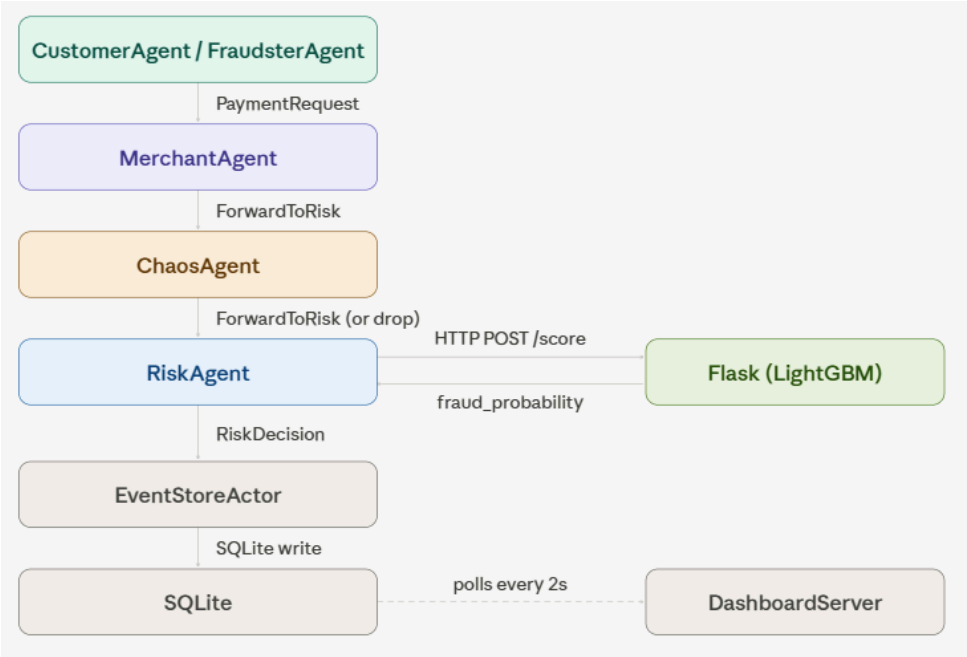
The simulation pipeline is written in Scala. The ML model and microservice are written in Python. The two communicate over HTTP; this keeps them fully independent. Flask can be retrained or replaced without touching the Scala codebase.

2. Architecture

2.1 Message protocol

Before any actor was built, the message types were defined first. In Akka Typed, an actor's behaviour is parameterised by the type of messages it can receive; the compiler enforces that no actor is ever sent a message it cannot handle. All message types live in Messages.scala.

Message	Fields	Sent by	Received by
PaymentRequest	id, cardId, amount, merchantId, merchantName, location	CustomerAgent, FraudsterAgent	MerchantAgent
ForwardToRisk	paymentRequest, timestamp, location	MerchantAgent	ChaosAgent → RiskAgent
RiskDecision	paymentId, outcome	RiskAgent	MerchantAgent, EventStoreActor



The diagram shows the full message flow for a single transaction from generation through to persistence.

2.2 Actor spawning order

Actors are spawned in dependency order inside Main.scala. Each actor is passed a reference to the actors it needs to communicate with at construction time.

#	Actor	Why this order
1	EventStoreActor	No dependencies — must exist before RiskAgent can reference it
2	RiskAgent	Needs a reference to EventStoreActor to send decisions
3	ChaosAgent	Needs a reference to RiskAgent to forward (or drop) messages
4	MerchantAgent	Needs a reference to ChaosAgent to forward payments
5	CustomerAgent	Needs a reference to MerchantAgent to send requests
6	FraudsterAgent	Needs a reference to MerchantAgent to send fraudulent requests

Concurrency note: Once spawned, all six actors run concurrently on Akka's internal thread pool. The spawning order only determines dependency resolution at startup. A single Akka dispatcher typically manages all actors across a pool of 8–16 threads, with each actor processing one message at a time.

The pipeline has three clear isolation boundaries that were deliberate design choices:

1. Scala / Python boundary. RiskAgent calls Flask over HTTP. The ML model is entirely isolated: it can be retrained, replaced, or restarted independently. If Flask is unavailable, RiskAgent falls back to rules-only scoring without crashing.
2. Pipeline / persistence boundary. EventStoreActor is the only actor that uses SQLite. All other actors are stateless - they send decisions and forget. If the write fails, the pipeline continues.

3. Pipeline / dashboard boundary. DashboardServer reads from SQLite directly. It has no connection to the actor pipeline, it cannot affect transaction processing, and the pipeline does not wait for the dashboard to acknowledge anything.

3. Agents & Components

3.1 CustomerAgent: Simulates legitimate payment traffic. Fires on a 2-second timer, randomly selects a card and merchant from hardcoded lists, and sends a PaymentRequest to MerchantAgent.

Property	Detail
Message in	Tick (internal timer)
Message out	PaymentRequest → MerchantAgent
Card pool	20 cards (CARD_01–CARD_20) - large pool so velocity is rarely tripped by legitimate use
Amount range	$50–$500 — deliberately capped below the high-amount rule threshold
Locations	Toronto, New York, London, Vancouver - normal cities only
Fire rate	Every 2 seconds

3.2 FraudsterAgent: Simulates compromised card activity. Designed to consistently trigger the rule engine's fraud detection signals - high velocity, high amounts, and flagged locations.

Property	Detail
Message in	Tick (internal timer)
Message out	PaymentRequest → MerchantAgent
Card pool	3 stolen cards (STOLEN_01–03) - small pool reused constantly to trip velocity check
Amount range	$600–$1000 - always above the $500 threshold
Locations	Lagos, Reykjavik, Tokyo - flagged as suspicious
Fire rate	Every 300ms. 6× faster than legitimate traffic

3.3 MerchantAgent: The routing layer between traffic sources and the risk engine. Receives every PaymentRequest, enriches it with a timestamp and location, and forwards it to ChaosAgent. Also handles deduplication - if the same payment ID arrives twice within 60 seconds (a result of ChaosAgent duplicating messages), the second copy is dropped silently.

Property	Detail
Message in	ReceivePayment(PaymentRequest), ReceiveDecision(RiskDecision)

Message out	ForwardToRisk → ChaosAgent
Internal state	seenIds: Map[String, Long] - payment ID → timestamp, pruned after 60 seconds
Enrichment	Adds System.currentTimeMillis() timestamp and location to ForwardToRisk
Deduplication	Same payment ID within 60s is dropped with a log warning

3.4 ChaosAgent: Injects realistic network failures between MerchantAgent and RiskAgent. Models the kinds of failures that occur in real payment infrastructure such as delayed responses, dropped connections, and duplicate submissions from retrying terminals.

Behaviour	Probability	What happens
Normal forward	60%	Message forwarded to RiskAgent immediately
Delay	20%	Thread sleeps 500ms–2s, then forwards
Drop	10%	Message silently discarded
Duplicate	10%	Same message sent to RiskAgent twice; MerchantAgent's dedup catches the second copy

Known limitation: ChaosAgent uses Thread.sleep for delays, which blocks the Akka dispatcher thread. This is a deliberate simplification and the correct approach is context.scheduleOnce, which releases the thread immediately and re-delivers the message later.

3.5 RiskAgent: The core decision-making actor. Every ForwardToRisk message is evaluated against a velocity check and a weighted scoring model, and optionally scored by the LightGBM ML model via Flask. The outcome: Approved, Declined, or HeldForReview is sent to EventStoreActor.

Property	Detail
Message in	Score(ForwardToRisk, replyTo)
Message out	RiskDecision → EventStoreActor
Internal state	recentPurchases: Map[String, List[Long]] - card ID → list of timestamps within 60s window
seenIds	Map[String, Long] — payment ID → timestamp, for deduplication within 60s
ML integration	HTTP POST to Flask on port 5001; falls back to rules-only if Flask is unavailable

The scoring model assigns points based on transaction signals. Both the rule score and ML probability are logged for every transaction, making disagreements visible.

Signal	Points added
Amount > $500	+30
Transaction between 11pm–5am	+25
Location anomaly flagged (15% random)	+25
Same card >3 purchases in 60 seconds	Auto-Declined (bypasses scoring)

Score	Outcome
< 40	Approved
40 – 69	HeldForReview
≥ 70	Declined

3.6 EventStoreActor: The only actor that writes to the database. Receives a RiskDecision and the original PaymentRequest for every scored transaction and inserts a row into SQLite. Also fires a 30-second summary timer printing approval rates and totals to the console. Decisions accumulate across restarts - the database is not wiped on each run.

Property	Detail
Message in	RecordEvent(PaymentRequest, RiskDecision), PrintSummary (timer)
Message out	None
Internal state	SQLite connection (JDBC), in-memory event list for summary stats
Database	events.db in the project root — created on first run
Summary timer	Fires every 30 seconds — prints approval rate, fraud rate, held count, total decisions

3.7 DashboardServer: A standalone Akka HTTP server started from Main.scala alongside the actor system. Reads from SQLite and serves the browser dashboard. Completely decoupled from the actor pipeline.

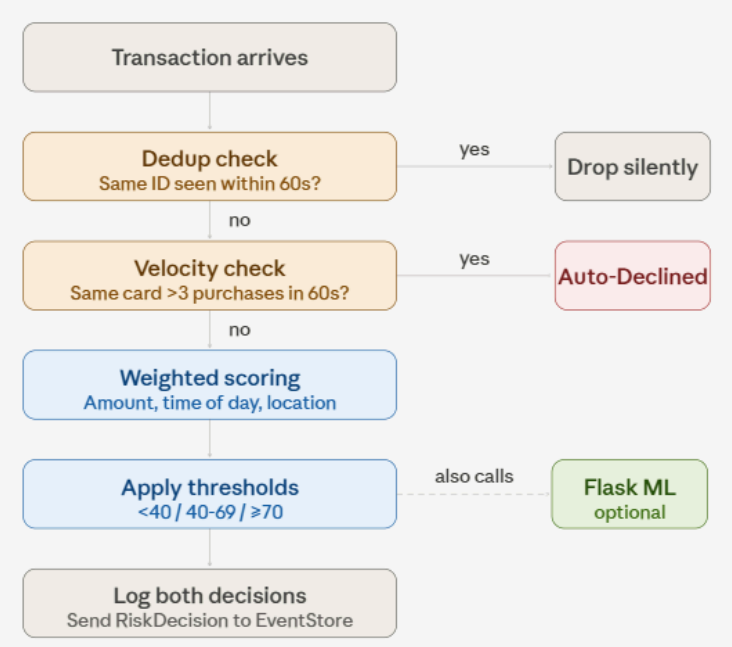
Endpoint	Returns
GET /	HTML dashboard page
GET /events	Last 20 transactions as JSON — polled every 2 seconds by the browser
GET /stats	Approval rate, fraud rate, held count, total volume

Design note: The dashboard polls /events every 2 seconds rather than using WebSockets. WebSockets would require a persistent connection and additional infrastructure on the server side. A 2-second poll is imperceptible to a human watching the dashboard and keeps the server implementation stateless.

4. Risk engine logic

Every transaction that reaches RiskAgent passes through two evaluation layers in sequence: a velocity check, then a weighted scoring model. The velocity check is a hard gate; if it fires, the transaction is declined immediately without scoring. The scoring model handles everything else.

Both the rule decision and the ML probability are logged for every transaction, making disagreements between the two systems directly observable in the dashboard and in SQLite.



The diagram below shows the exact sequence RiskAgent follows for each incoming message.

4.1 Velocity check

The velocity check runs first on every transaction. If the same card ID has made 3 or more purchases within the last 60 seconds, the transaction is declined immediately - the scoring model is not consulted.

Implementation: RiskAgent maintains a Map[String, List[Long]] - card ID mapped to a list of recent timestamps. On every transaction, timestamps older than 60 seconds are pruned from the list before the check. This keeps memory bounded without a separate cleanup process, and the pruning happens inline as part of normal message processing.

Parameter	Value	Rationale
Window	60 seconds	Covers burst fraud patterns without penalising normal repeat customers
Threshold	3 purchases	Low enough to catch FraudsterAgent's 300ms fire rate within seconds
Override	Yes — bypasses scoring	Velocity is a strong enough signal to decline without further evaluation

4.2 Weighted scoring model

Transactions that pass the velocity check are scored against three signals. Points are additive - a transaction can trigger multiple signals and their points stack.

Signal	Condition	Points
High amount	Transaction amount exceeds $500	+30
Late night	Transaction hour is 11pm–5am (America/Toronto timezone)	+25
Location anomaly	15% random probability per transaction — simulates an unknown or flagged location signal	+25

4.3 State management in RiskAgent

RiskAgent carries two pieces of state across messages - neither uses mutable variables.

State	Type	Purpose
recentPurchases	Map[String, List[Long]]	Card ID → list of recent timestamps. Used for velocity checking. Pruned on every message.
seenPayments	Map[String, Long]	Payment ID → first-seen timestamp. Used for deduplication. Entries expire after 60 seconds.

5. ML model

The ML layer adds a second opinion alongside the rule engine. A LightGBM classifier is trained on synthetic transaction data and served by a Flask microservice. For every transaction that reaches RiskAgent, the model returns a fraud probability between 0 and 1. Both the rule decision and the ML probability are written to SQLite and shown side by side in the dashboard. The model does not make the final binding decision - the rule engine does.

5.1 Synthetic training data

generate_data.py produces 10,000 labelled transactions. The card ID is never included as a feature - the model learns from behaviour patterns only. This ensures the classifier cannot simply memorise "STOLEN = fraud".

Feature	Description
amount	Transaction amount in USD
velocity	Number of purchases from this card in the last 60 seconds
location_risk	1 if location is Lagos, Reykjavik, or Tokyo — 0 otherwise
hour	Hour of day (0–23, America/Toronto timezone)
is_high_risk_merchant	1 for high-fraud merchants (e.g. Amazon) — 0 otherwise

The dataset contains three transaction categories, with intentionally overlapping patterns to prevent the model from learning overly clean boundaries:

Category	Count	Characteristics
Legitimate	8,000	Low velocity (1–2/60s), daytime hours, amounts $50–$500. 10% include travel to flagged locations to add realistic overlap.
Classic fraud	1,400	High velocity (4–15/60s), high amounts ($400–$1,000), flagged locations. Designed to be clearly detectable.
Sneaky fraud	600	Low velocity, normal amounts, safe locations — deliberately designed to look legitimate. Tests the model's ability to find subtle signals.

Why sneaky fraud? A model trained only on obvious fraud patterns would perform artificially well. The 600 sneaky fraud rows create a realistic hard case — low-profile fraud that evades velocity and location signals. The model's ability to partially catch these (without rules to lean on) is a genuine test of what it has learned.

5.2 Model performance

The classifier is trained on 8,000 rows and evaluated on a held-out test set of 2,000 rows.

Metric	Value	Interpretation
Overall accuracy	94%	Correct on 94 of every 100 transactions
Fraud precision	97%	When the model flags fraud, it is almost always correct — very few false positives
Fraud recall	71%	The model catches 71% of fraud — 29% slips through, mostly the sneaky fraud category
Legit recall	99%	Almost no legitimate transactions are incorrectly flagged

5.3 Flask microservice

flask_server.py loads fraud_model.pkl once at startup and exposes a single endpoint. The server runs on port 5001 and must be started before the Scala simulator.

Property	Detail
Endpoint	POST /score
Input	JSON — amount, velocity, location_risk, hour, is_high_risk_merchant
Output	JSON — fraud_probability (0.0–1.0), is_fraud (true/false)
Threshold	fraud_probability ≥ 0.5 → is_fraud: true
Port	5001

5.5 Integration with RiskAgent

RiskAgent calls Flask using Java 11's built-in HttpClient - no additional Scala dependency is required. The call is made synchronously after the rule decision is computed. If Flask is unavailable for any reason, callFlask() returns None and the system falls back to rules-only scoring without crashing.

- RiskAgent computes the rule decision first — independently of Flask. The rule decision is never overridden by the ML result.
- Flask is called once per transaction — velocity count and location risk flag are derived locally in RiskAgent before the HTTP call.
- Both results are logged — the console shows the ML probability and the rule decision side by side for every scored transaction.

Design note: why Flask over embedding the model in Scala? LightGBM is a Python library. Embedding it in Scala would require JNI bindings or rewriting the model in a JVM-compatible library - both are fragile and tightly coupled. A Flask microservice keeps the two languages independent: Flask can be retrained or replaced without touching the Scala codebase, and the Scala simulator does not need to know anything about how the model works internally.

6. Data layer

Every payment decision is written to a local SQLite database (events.db) by EventStoreActor. SQLite was chosen over a server-based database because it requires no installation or configuration. The database is the shared read/write surface between the actor pipeline and the dashboard. The pipeline writes to it; the dashboard reads from it. They never communicate directly.

6.1 Schema

Column	Notes
payment_id	UUID generated by CustomerAgent or FraudsterAgent.
outcome	Stored as a plain string. Either Approved, Declined, or HeldForReview.
ml_probability	Fraud probability returned by Flask (0.0–1.0). Stored as -1.0 if Flask was unavailable when the transaction was scored.
timestamp	Unix epoch in milliseconds — set by MerchantAgent when it wraps the PaymentRequest into ForwardToRisk.

EventStoreActor is the only actor that opens a database connection. All writers go through it. No other actor touches SQLite.

DashboardServer and EventStoreActor both read from SQLite independently. DashboardServer does this every time the browser polls /events (every 2 seconds). EventStoreActor does this every 30 seconds to print the console summary.

Because decisions are written to disk rather than held in memory, the event log survives the process restarts. Restarting the simulator appends new decisions to the existing database and the total count in the 30-second summary reflects all decisions made across all runs, not just the current session.

7. Extending the system

The system was designed with clear isolation boundaries - each component has a single responsibility and communicates through well-defined interfaces. This makes it straightforward to extend individual layers without rebuilding others. The natural next steps below are grouped by the layer they touch.

1. Akka Cluster Sharding — distribute RiskAgent across nodes: Convert RiskAgent to a cluster-sharded actor, where each card ID routes consistently to one shard. Spin up 2–3 local cluster nodes and watch traffic distribute automatically.

2. Model retraining pipeline using live SQLite data: The simulator already generates labelled ground truth - every transaction stored in SQLite has a card ID that identifies whether it was fraud. A retraining script could retrain the LightGBM model on accumulated live data, and hot-swap the model in Flask without restarting the simulator. The feedback loop closes: the model improves as the system runs.

3. Open-source the data stream for external model benchmarking: Since the simulator controls ground truth — STOLEN cards are always fraud — the transaction stream is a labelled dataset. Exposing it via a Kafka topic or a streaming HTTP endpoint would let other developers plug in their own fraud detection models and benchmark against the built-in rules and LightGBM baseline. The simulator becomes a testing harness, not just a demo.